

Modern Multi-Agent Coding Architectures

<https://gemini.google.com/app/dc3c4ab9fa6a5a04>

User prompt: What would be the best, and most modern way, of having my coding agents collaborate with each other on a project? For them to actually communicate, discuss ideas, delegate tasks, and continue creating and generating

Response: The most modern approach to multi-agent coding has shifted away from trying to make one monolithic LLM do everything, moving toward stateful orchestration where specialized agents communicate through a shared state or message graph.

Here is a look at how the routing actually flows depending on the architecture you choose.

When building orchestration proxies and toolchains, your choice of framework dictates how these agents actually collaborate.

Framework	Core Mechanism	Best For
LangGraph	Graph-based state machines	Strict, cyclical workflows (e.g., Code -> Test -> Review -> Loop).
AutoGen	Conversational agents	Open-ended brainstorming and flexible, peer-to-peer task delegation.
CrewAI	Role-based hierarchy	Setting up a "virtual team" where a top-level Manager delegates down.
Google Antigravity	Native environment access	Building and deploying deterministic agent pipelines directly to infrastructure.

To make a multi-agent system reliable enough for production work, the architecture requires a few specific design patterns:

Decoupled Memory and State Agents need a shared workspace to read and write context. Instead of relying on isolated or fragmented memory tools, modern setups use framework-native state check-pointing or standard vector databases (like Qdrant or Postgres with pgvector) as a unified long-term memory layer for the whole group. The state acts as the "whiteboard" everyone looks at.

Narrow Role Prompting A "Developer" agent should only be given tools to read/write files and access terminal commands. A "Reviewer" agent only gets diff-reading tools and read-only access. Strictly siloing their tools and system prompts prevents agents from overwriting each other's work or drifting out of their lane.

Sandboxed Execution Because these agents are generating and executing code, their environments need to be isolated. Tying your orchestration into transient Docker containers or dedicated Kubernetes pods ensures agents can compile, test, and fail safely before any changes are finalized.

Human-in-the-Loop Routing A router proxy sits between the agents and you. If the QA agent detects a failing test it can't resolve after a set number of execution loops, the system pauses, snapshots the current state, and escalates the context to you for a final decision.

User prompt: yes

Response: Setting up a reliable state-sharing loop requires treating the state object as an append-only ledger and isolating the actual code execution. This prevents the Developer from hallucinating over its own instructions and strictly prohibits the Reviewer from directly rewriting the codebase.

1. Define a Strict State Schema Use a strongly typed structure (like Pydantic in Python or Zod in TypeScript) to act as the shared memory graph. Do not let agents pass raw conversational text strings back and forth.

```
from typing import List, TypedDict

class AgentState(TypedDict):
    task_description: str
    current_code: str
    file_path: str
    reviewer_feedback: List[str] # Append-only list
    iteration_count: int
    status: str # "in_progress", "pending_review", "approved"
```

2. Isolate the Workspaces Instead of agents writing directly to your local file system, mount a transient directory inside a Docker container or a dedicated Kubernetes pod. The Developer writes its output to this isolated sandbox, and the Reviewer executes compilation or testing tools against that exact mounted directory.

3. Restrict Agent Permissions Siloing their tools ensures they stay in their lanes:

- **Developer Node:** Given tools to read the `task_description` and the latest item in `reviewer_feedback`, and a tool to write code to the sandbox. Once done, it updates the state's `current_code` string, increments the `iteration_count`, and changes the status to `pending_review`.
- **Reviewer Node:** Given read-only access to the sandbox files and execute-only access to terminal commands (like `pytest` or `go test`). It physically cannot edit the code. It appends its findings to `reviewer_feedback` and updates the status to `in_progress` (if tests fail) or `approved` (if tests pass).

4. Build the Conditional Routing Loop Using your orchestration layer, establish the conditional edges that move the state between the agents based entirely on the status flag.

```
# Conceptual graph routing logic
def route_workflow(state: AgentState):
    if state["iteration_count"] > 5:
        return "escalate_to_human_proxy"

    if state["status"] == "pending_review":
        return "Reviewer_Agent"

    if state["status"] == "in_progress":
        return "Developer_Agent"

    if state["status"] == "approved":
        return "Finalize_Merge"
```

With this architecture, the Developer only acts on the Reviewer's structured feedback array, not its raw "thought" process. The Reviewer only evaluates the executed code output, not the Developer's intermediate drafts. If they hit the iteration limit without reaching an approved status, the orchestration pauses and passes the state to your CLI proxy for manual intervention.

User prompt: How do I safely implement the Docker or Kubernetes sandboxed execution environment for these agents?

Response: Implementing a secure sandbox means treating the agent's generated code as an untrusted third-party payload. The safest pattern is to have your orchestration proxy spin up ephemeral, heavily restricted containers via the Docker SDK or Kubernetes API, execute the code remotely, and immediately destroy the environment.

1. Dynamic Provisioning Don't use a long-running container. Your orchestrator should provision a fresh Pod or Docker container for each distinct task or PR. This ensures no residual state corrupts the next test run.

```
# Pseudo-code for orchestrator using kubernetes Python client
from kubernetes import client, config

def create_sandbox_pod(task_id):
    pod_manifest = {
        "apiVersion": "v1",
        "kind": "Pod",
        "metadata": {"name": f"agent-sandbox-{task_id}"},
        "spec": {
            "containers": [{
                "name": "runner",
                "image": "python:3.11-slim", # Or your specific build image
                "command": ["sleep", "3600"], # Keep alive for exec
                "securityContext": {
                    "runAsNonRoot": True,
                    "runAsUser": 1000,
                    "allowPrivilegeEscalation": False,
                    "readOnlyRootFilesystem": True
                },
            },
            "volumeMounts": [{"mountPath": "/workspace", "name": "workdir"}]
        },
        "volumes": [{"name": "workdir", "emptyDir": {}}]
    }
    # Create pod via K8s API
```

2. The Execution Interface Agents never get SSH access. When the Developer agent writes a file, the orchestrator injects that file directly into the pod's `/workspace` volume. When the Reviewer agent needs to test the code, the orchestrator triggers an API-level execution and pipes the output back to the state graph.

```
# The orchestrator executes the command inside the running pod
from kubernetes.stream import stream

def run_tests_in_sandbox(pod_name, command=["pytest", "/workspace/tests"]):
    response = stream(
        core_v1_api.connect_get_namespaced_pod_exec,
        pod_name,
        'default',
        command=command,
        stderr=True, stdin=False, stdout=True, tty=False
    )
    return response # This string goes to the Reviewer's feedback state
```

3. Network Isolation If an agent hallucinates a malicious package or writes code that attempts to scrape local endpoints, you need egress controls. In Kubernetes, apply a `NetworkPolicy` to the sandbox namespace that completely blocks internal cluster access (preventing SSRF) and only allows egress to specific package registries (like npm or PyPI) if dynamic dependency installation is required.

4. Ephemeral Teardown Once the `iteration_count` hits its limit or the status reaches `approved`, the orchestrator issues a delete command for the pod. The `emptyDir` volume evaporates, taking any generated artifacts or temporary state with it.

User prompt: yes

Response: Building the human-in-the-loop escalation requires setting a hard breakpoint in your agent graph and handing the state payload over to a local terminal process. You can pipe the interrupt directly to your custom CLI router or a zsh script to handle the intervention without breaking your workflow.

Here is how the handoff works in practice.

1. Set the Graph Interrupt In your orchestration framework, you define a node specifically for human escalation. When the routing logic hits a threshold—like `iteration_count > 5` or a specific critical failure from the Reviewer agent—the graph yields execution.

```
# The routing logic triggers the interrupt
def route_workflow(state: AgentState):
    if state["iteration_count"] >= 5:
        # Route to the breakpoint node
        return "Human_Proxy_Node"

    if state["status"] == "pending_review":
        return "Reviewer_Agent"
    # ...
```

2. State Serialization (The Handoff) When execution pauses at the `Human_Proxy_Node`, the orchestrator serializes the current state. Instead of overcomplicating it with memory abstraction layers like `mem0`, you just dump the state dictionary directly to a local SQLite database or an IPC socket that your CLI tool monitors.

3. The CLI Listener You run a local script in your terminal that polls for these serialized state payloads. When an escalation hits, it pulls the data and renders the context, showing you exactly where the agents got stuck.

```
def cli_proxy_listen():
    pending_state = check_local_db_for_escalations()

    print(f"Graph paused at iteration {pending_state['iteration_count']}")
    print("Latest Reviewer Error:")
    print(pending_state['reviewer_feedback'][-1])

    # Present options directly in the terminal
    action = input("\nAction: (1) Send hint to Developer (2) Approve manually (3) Abort >> ")

    if action == "1":
        hint = input("Enter hint: ")
        # Inject your instruction as the highest-priority feedback
        pending_state["reviewer_feedback"].append(f"SYSTEM OVERRIDE: {hint}")
        pending_state["status"] = "in_progress"

    elif action == "2":
        pending_state["status"] = "approved"
```

```
# Send the modified state back to the orchestrator  
resume_graph_execution(pending_state)
```

4. Graph Resumption Once you submit your choice, the CLI proxy writes the updated state back to the orchestrator and clears the local queue. The orchestration framework picks up the new state object and resumes execution right where it left off. If you selected option 1, the graph routes back to the Developer agent, which now reads your manual hint as the next step in its instruction array.

This setup keeps your agents entirely sandboxed during their execution loops, while giving you a clean terminal interface to unblock them the moment they start spinning their wheels.
